SIMATS ENGINEERING

ASSESSMENT TOOL II – DATA ANALYSIS PROJECT

Course: Data Handling and Visualization	**Code: DSA0601**	**CO:** CO3
Duration: 120 min	**Total Marks:** 100	Reg No : 192524108 Name: S. LAKSHITA

COURSE OUTCOME COVERED

CO3: Analyze time-series data, trends, associations, and uncertainty through appropriate visualization methods. (BL4)

Activity Tool : Data Analysis Project

Weightage: 20%

Code of Conduct:

I S.LAKSHITA(192524108) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criteria	Problem Understanding & Data Preparation 25%	Data Analysis & Interpretation 30%	Application of Analytical Techniques 20%	Analysis of Results 15%	Organization & Documentation 10%	Presentation & Clarity 5%	Total
Q1							
Q2							
Q3							
Q4							
Q5							

Data Analysis Project

Question 1 – Distribution Analysis Using ECDF And Q-Q Plot

Dataset: Employee_Salaries.csv

Employee_ID	Department	Salary
E001	IT	42000
E002	IT	45000
E003	IT	48000
E004	IT	51000
E005	IT	55000
E006	HR	38000
E007	HR	40000
E008	HR	42000
E009	HR	46000
E010	HR	50000
E011	Finance	47000
E012	Finance	49000
E013	Finance	52000
E014	Finance	58000
E015	Finance	65000
E016	Marketing	36000
E017	Marketing	39000
E018	Marketing	43000
E019	Marketing	47000
E020	Marketing	72000

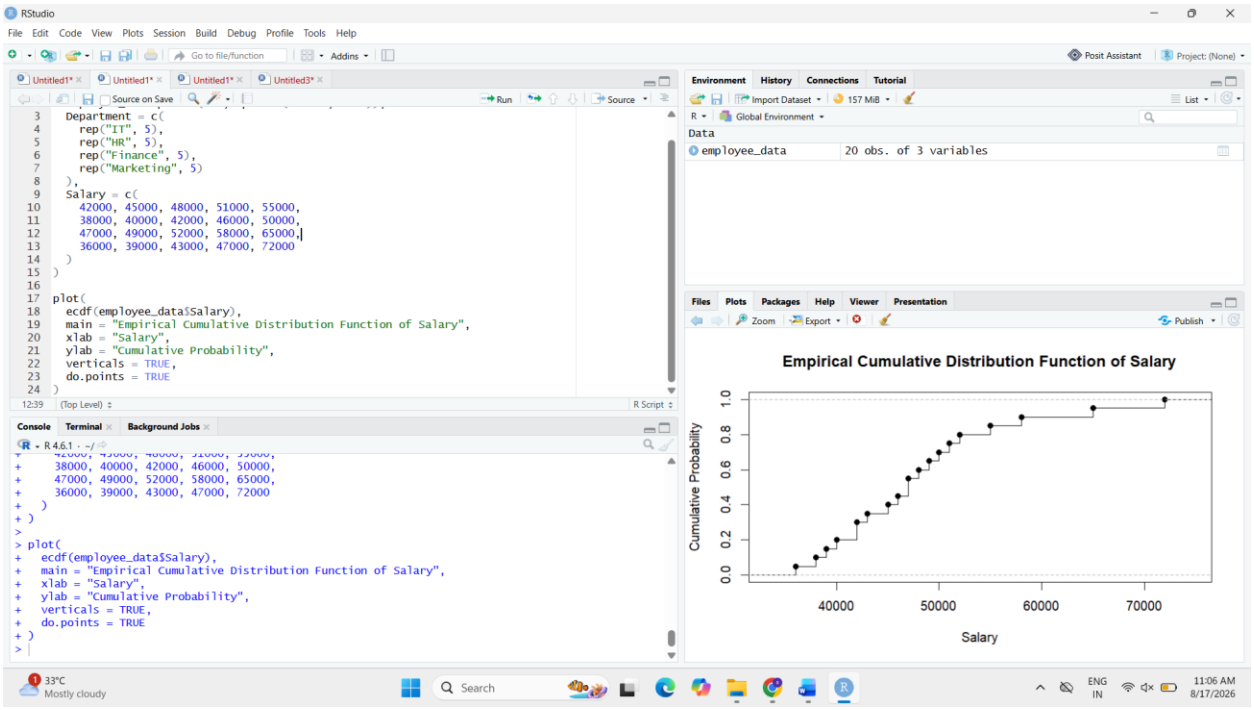
1. Create an empirical cumulative distribution function (ECDF) for Salary.

```
employee_data <- data.frame(
  Employee_ID = paste0("E", sprintf("%03d", 1:20)),
  Department = c(
    rep("IT", 5), rep("HR", 5),
    rep("Finance", 5), rep("Marketing", 5)
  ),
  Salary = c(
    42000, 45000, 48000, 51000, 55000,
    38000, 40000, 42000, 46000, 50000,
    47000, 49000, 52000, 58000, 65000,
    36000, 39000, 43000, 47000, 72000
  )
)
plot(
  ecdf(employee_data$Salary),
  main = "Empirical Cumulative Distribution Function of Salary",
  xlab = "Salary",
  ylab = "Cumulative Probability",
  verticals = TRUE,
  do.points = TRUE
)
```

Interpretation:

The ECDF shows the cumulative proportion of employees earning salaries up to

each salary value. Salaries range from ₹36,000 to ₹72,000, with most salaries concentrated between ₹38,000 and ₹58,000.

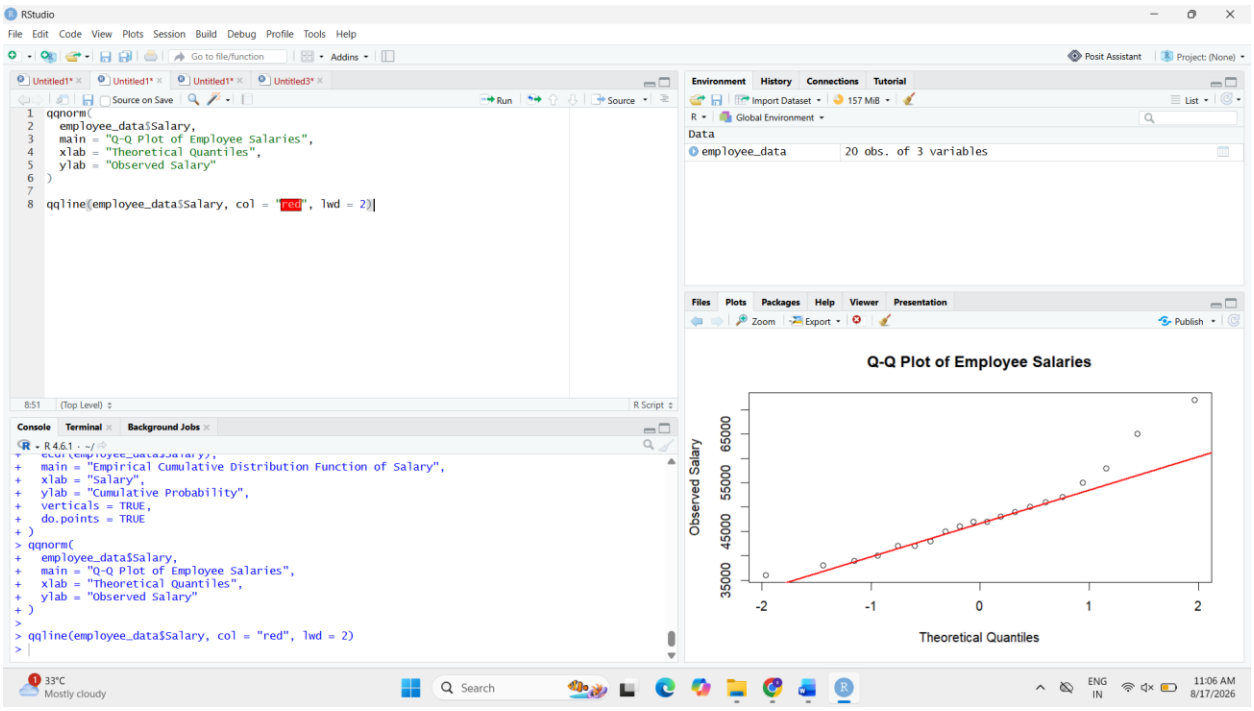


2. Create a Q-Q plot to assess whether Salary is approximately normally distributed.

```
qqnorm(
  employee_data$Salary,
  main = "Q-Q Plot of Employee Salaries",
  xlab = "Theoretical Quantiles",
  ylab = "Observed Salary"
)

qqline(employee_data$Salary, col = "red", lwd = 2)
```

Interpretation:

The points show deviation from the reference line, especially at the upper end. Therefore, the salary distribution is not perfectly normally distributed.



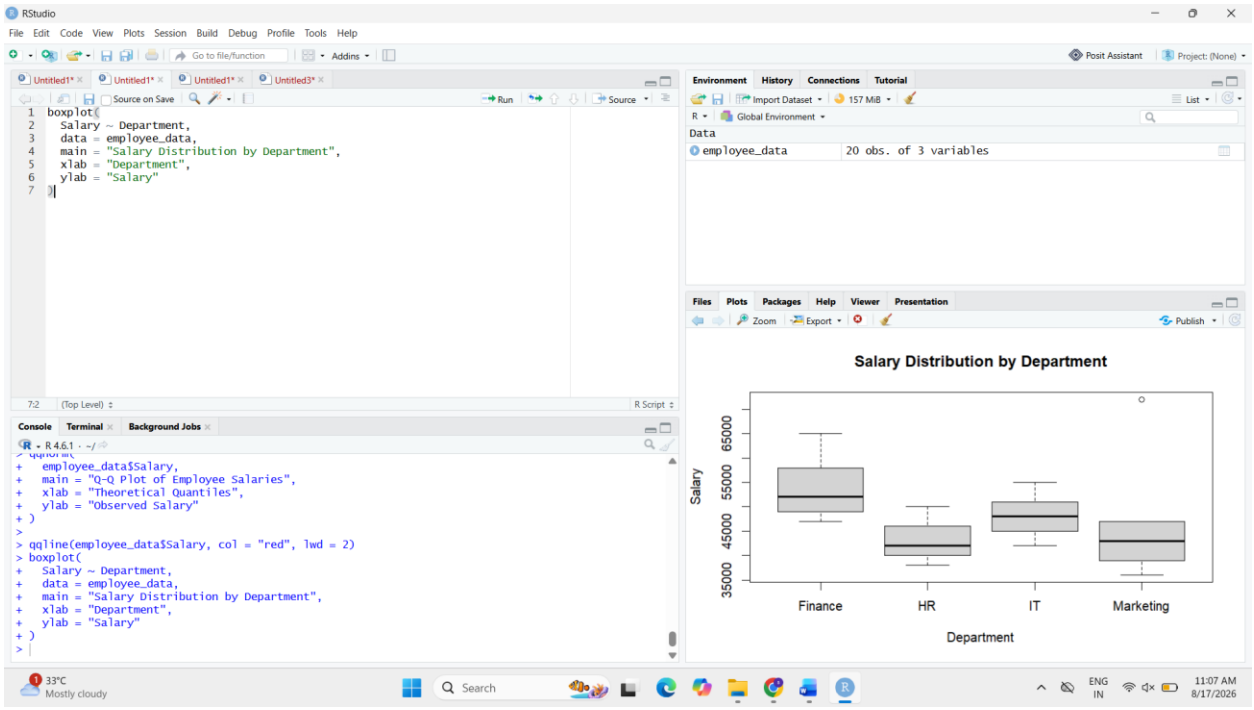
3. Compare the salary distributions of the four departments using suitable distribution plots.

```
boxplot(
  Salary ~ Department,
  data = employee_data,
  main = "Salary Distribution by Department",
  xlab = "Department",
  ylab = "Salary"
)
```

Interpretation:

- **IT:** ₹42,000–₹55,000
- **HR:** ₹38,000–₹50,000
- **Finance:** ₹47,000–₹65,000

- **Marketing:** ₹36,000–₹72,000 and has the widest spread.



4. Identify any possible skewness or unusual observations and explain what the plots reveal.

The salary distribution shows **right skewness** because of the higher salary values of ₹65,000 and ₹72,000. The **₹72,000 Marketing salary** is particularly unusual and may be considered a potential high-end outlier.

Marketing has the greatest salary variation, while HR has comparatively lower salaries.

Question 2 – Visualizing Many Distributions Using Bean Plots

Dataset: Student_Performance.csv

Student_ID	Course	Study_Hours	Score
S001	Python	5	62
S002	Python	7	68

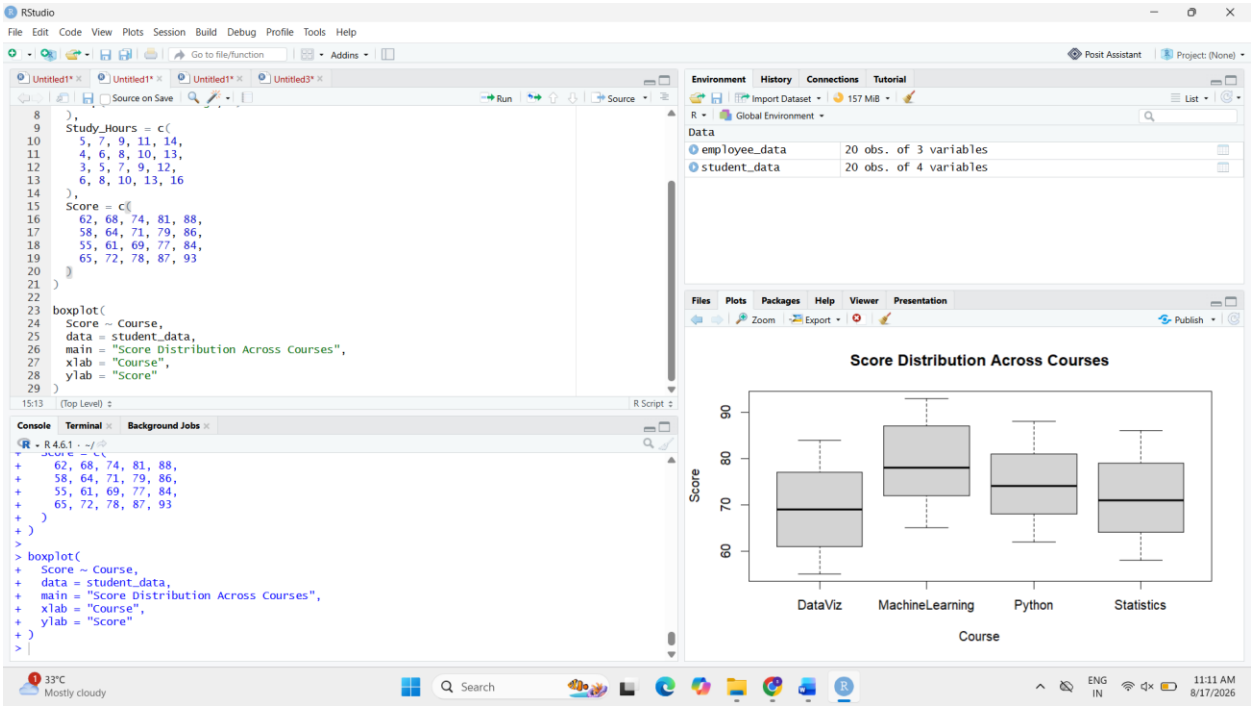
S003	Python	9	74
S004	Python	11	81
S005	Python	14	88
S006	Statistics	4	58
S007	Statistics	6	64
S008	Statistics	8	71
S009	Statistics	10	79
S010	Statistics	13	86
S011	DataViz	3	55
S012	DataViz	5	61
S013	DataViz	7	69
S014	DataViz	9	77
S015	DataViz	12	84
S016	MachineLearning	6	65
S017	MachineLearning	8	72
S018	MachineLearning	10	78
S019	MachineLearning	13	87
S020	MachineLearning	16	93

1. Visualize the score distribution for all four courses at the same time.

```
student_data <- data.frame(
  Student_ID = paste0("S", sprintf("%03d", 1:20)),
  Course = c(
```

```
    rep("Python", 5),
    rep("Statistics", 5),
    rep("DataViz", 5),
    rep("MachineLearning", 5)
  ),
  Study_Hours = c(
    5, 7, 9, 11, 14,
    4, 6, 8, 10, 13,
    3, 5, 7, 9, 12,
    6, 8, 10, 13, 16
  ),
  Score = c(
    62, 68, 74, 81, 88,
    58, 64, 71, 79, 86,
    55, 61, 69, 77, 84,
    65, 72, 78, 87, 93
  )
)

boxplot(
  Score ~ Course,
  data = student_data,
  main = "Score Distribution Across Courses",
  xlab = "Course",
```

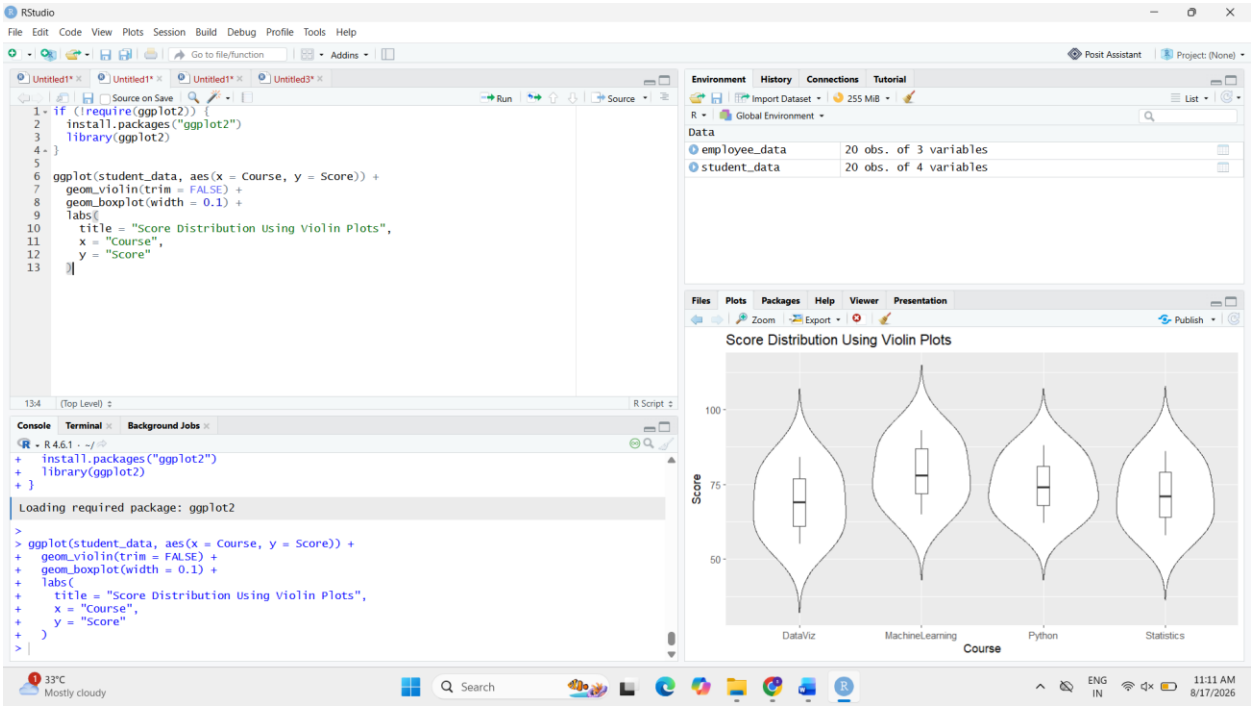
```
  ylab = "Score"
)
```



2. Create bean plots or an equivalent violin/density-based visualization.

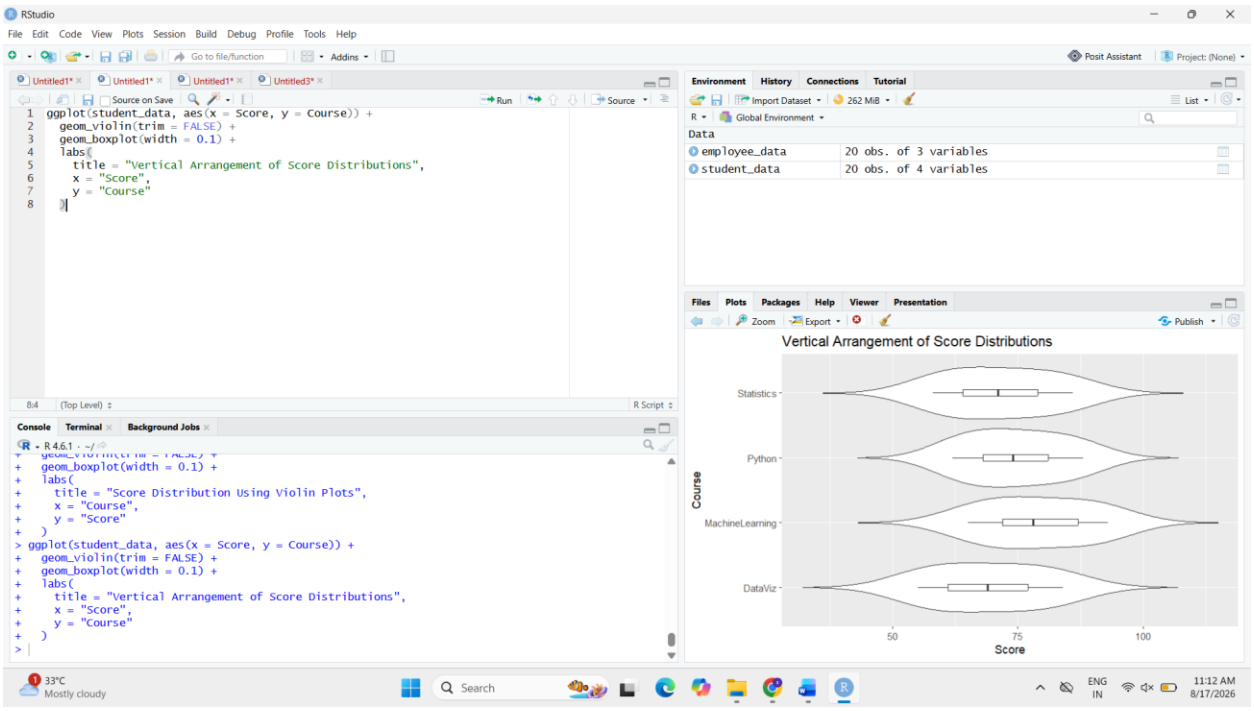
```
if (!require(ggplot2)) {
  install.packages("ggplot2")
  library(ggplot2)
}


ggplot(student_data, aes(x = Course, y = Score)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = 0.1) +
  labs(
    title = "Score Distribution Using Violin Plots",
    x = "Course",
```

```
  y = "Score"
)
```



3. Create a second visualization with the distributions arranged along the vertical axis.

```
ggplot(student_data, aes(x = Score, y = Course)) +
  geom_violin(trim = FALSE) +
  geom_boxplot(width = 0.1) +
  labs(
    title = "Vertical Arrangement of Score Distributions",
    x = "Score",
    y = "Course"
  )
```



4. Compare the center, spread, and shape of the distributions and determine which course has the highest and most consistent performance.

Comparison:

- **Python:** Scores range from 62 to 88, with a moderate spread.
- **Statistics:** Scores range from 58 to 86, with a moderate spread.
- **DataViz:** Scores range from 55 to 84 and has a relatively wide spread.
- **MachineLearning:** Scores range from 65 to 93 and has the highest overall scores.

Highest performance: MachineLearning, with the highest scores overall and a maximum score of **93**.

Most consistent performance: Python, as its scores show the smallest overall range among the four courses (**26 points**).

Conclusion:
MachineLearning demonstrates the strongest overall performance, while Python shows comparatively consistent scores. DataViz has the lowest starting score and a relatively wider distribution.

Question 3 – Comparing Proportions with Pie, Side-by-Side and Stacked Bars

Dataset: Customer_Purchases.csv

Customer_ID	Region	Product	Purchase_Type	Count
C001	North	Laptop	Online	25
C002	North	Phone	Online	35
C003	North	Tablet	Store	20
C004	South	Laptop	Store	30
C005	South	Phone	Online	40
C006	South	Tablet	Store	25
C007	East	Laptop	Online	28
C008	East	Phone	Store	32
C009	East	Tablet	Online	18
C010	West	Laptop	Store	22
C011	West	Phone	Online	38
C012	West	Tablet	Store	27

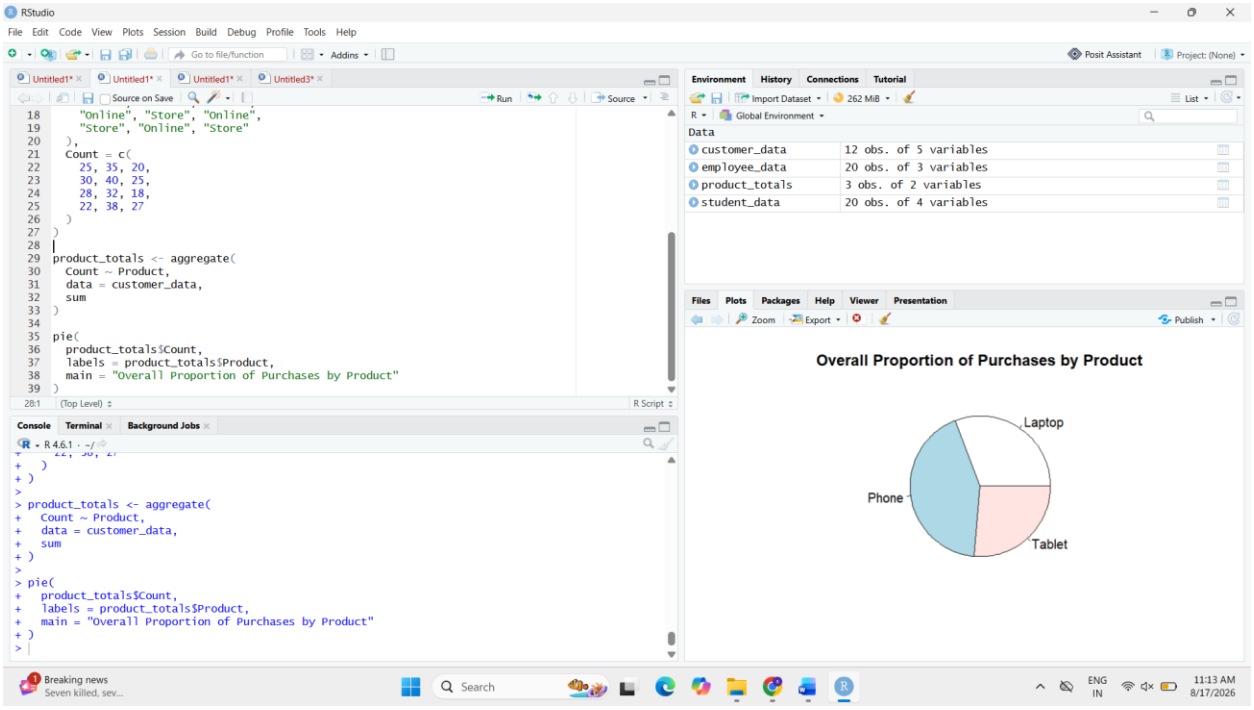
1. Create a pie chart showing the overall proportion of purchases by Product.

```
customer_data <- data.frame(
  Customer_ID = paste0("C", sprintf("%03d", 1:12)),
  Region = c(
    "North", "North", "North",
    "South", "South", "South",
    "East", "East", "East",
    "West", "West", "West"
```

```
  ),
  Product = c(
    "Laptop", "Phone", "Tablet",
    "Laptop", "Phone", "Tablet",
    "Laptop", "Phone", "Tablet",
    "Laptop", "Phone", "Tablet"
  ),
  Purchase_Type = c(
    "Online", "Online", "Store",
    "Store", "Online", "Store",
    "Online", "Store", "Online",
    "Store", "Online", "Store"
  ),
  Count = c(
    25, 35, 20,
    30, 40, 25,
    28, 32, 18,
    22, 38, 27
  )
)

product_totals <- aggregate(
  Count ~ Product,
  data = customer_data,
```

```
  sum
)

pie(
  product_totals$Count,
  labels = product_totals$Product,
  main = "Overall Proportion of Purchases by Product"
)
```

Interpretation:
Phones have the highest overall purchase count, followed by Laptops and Tablets. Therefore, **Phone** represents the largest overall proportion of purchases.



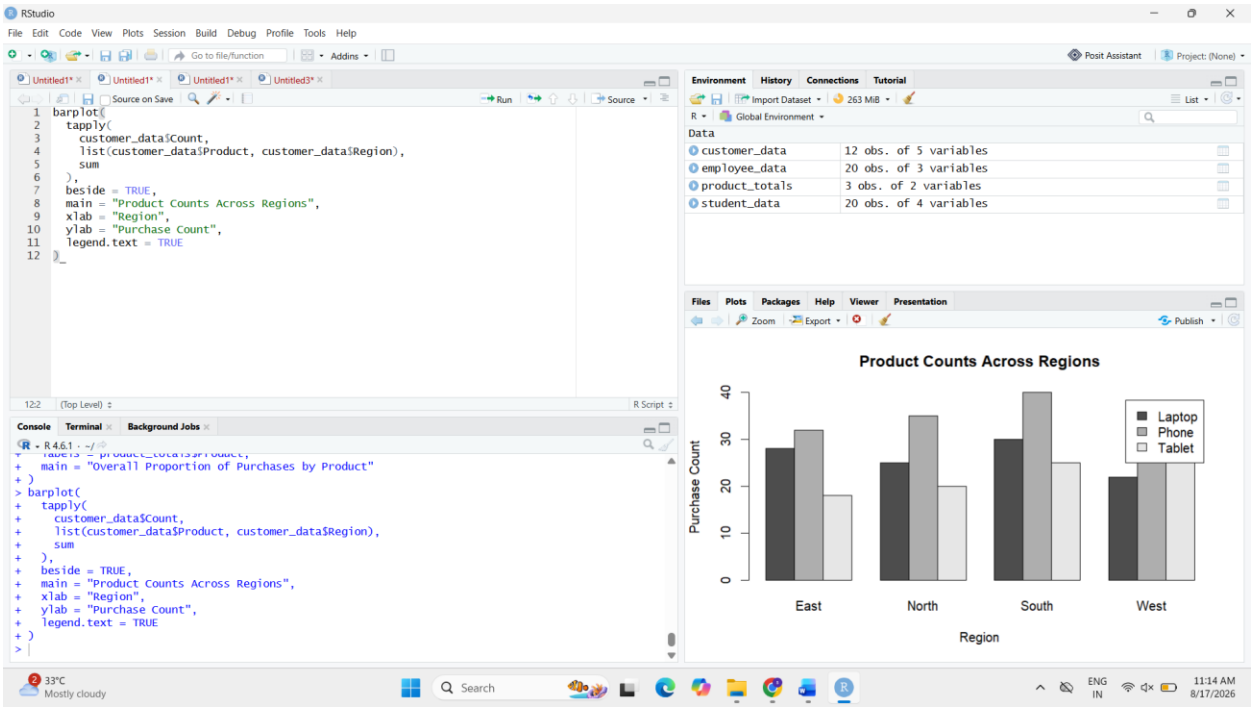
2. Create side-by-side bar charts comparing Product counts across Regions.

```
barplot(
  tapply(
    customer_data$Count,
```

```
  list(customer_data$Product, customer_data$Region),
  sum
 ),
 beside = TRUE,
 main = "Product Counts Across Regions",
 xlab = "Region",
 ylab = "Purchase Count",
 legend.text = TRUE
)
```

Interpretation:

The side-by-side bar chart allows direct comparison of the purchase counts of Laptop, Phone and Tablet within each region.



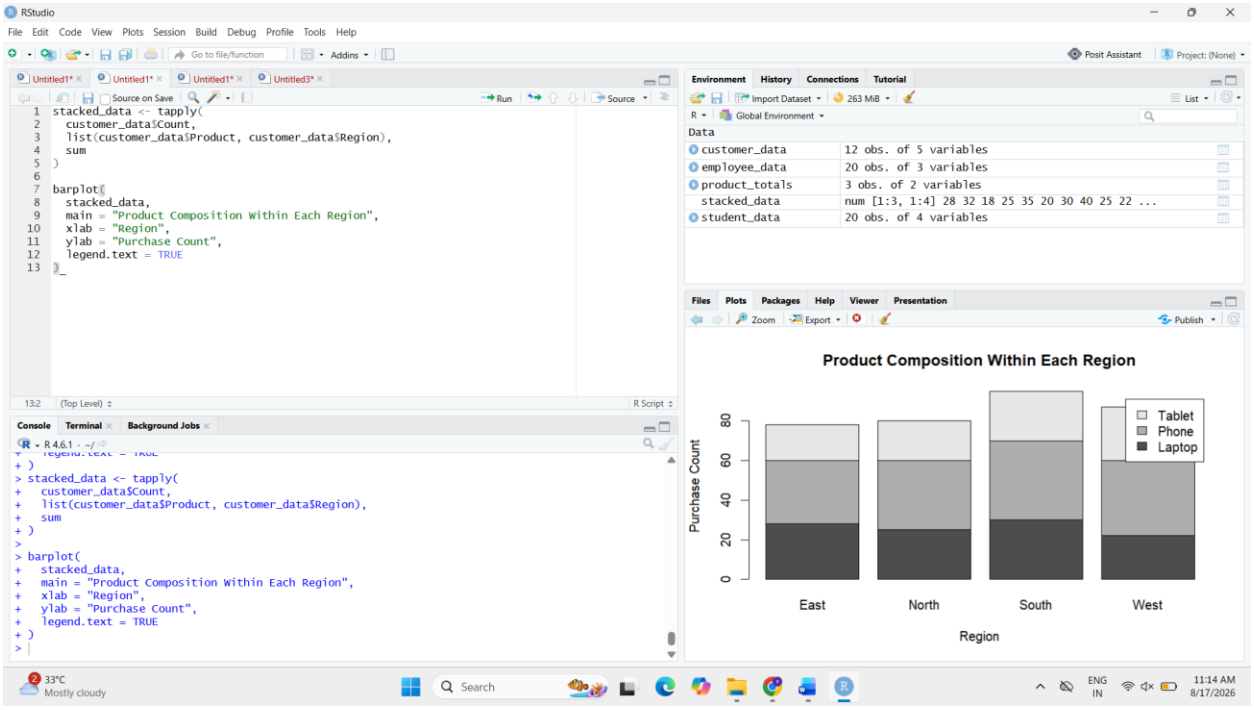
3. Create a stacked bar chart showing product composition within each Region.

```
stacked_data <- tapply(
```

```
  customer_data$Count,
  list(customer_data$Product, customer_data$Region),
  sum
)

barplot(
  stacked_data,
  main = "Product Composition Within Each Region",
  xlab = "Region",
  ylab = "Purchase Count",
  legend.text = TRUE
)
```

Interpretation:

The stacked bar chart displays the total purchases in each region while showing how each total is composed of different products.

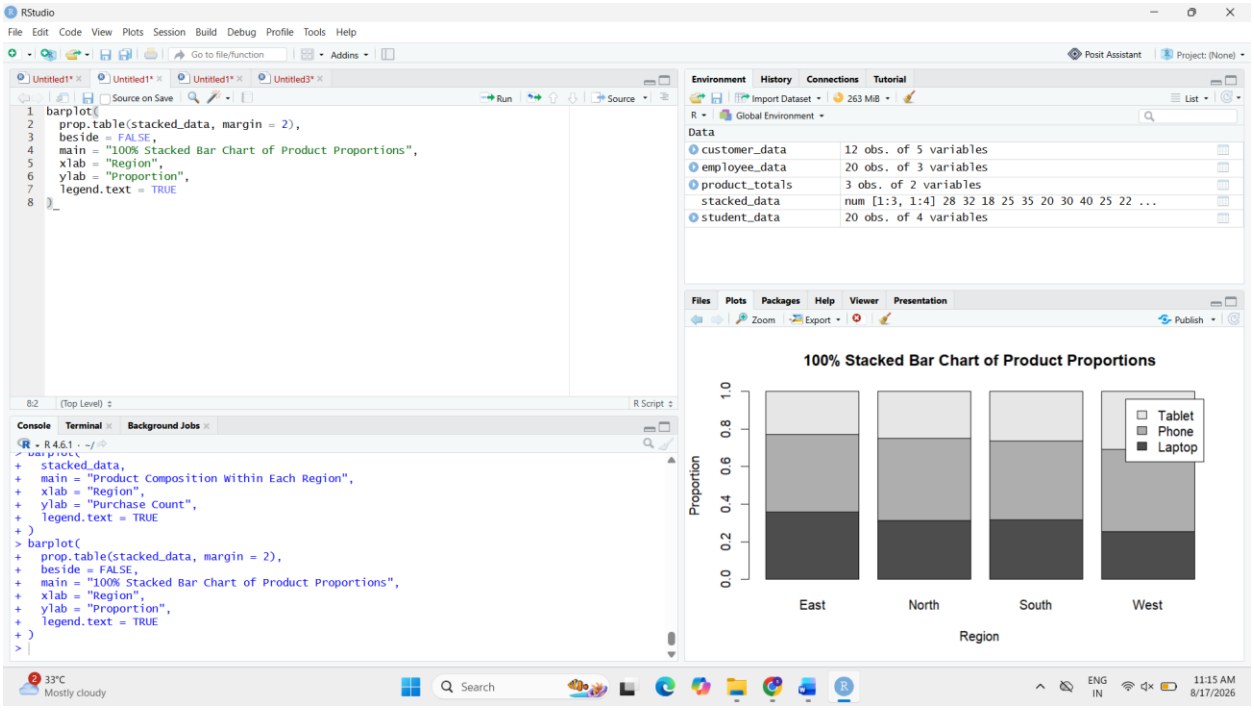


4. Convert the stacked chart into a 100% stacked bar chart and compare proportions rather than raw counts.

```
barplot(
  prop.table(stacked_data, margin = 2),
  beside = FALSE,
  main = "100% Stacked Bar Chart of Product Proportions",
  xlab = "Region",
  ylab = "Proportion",
  legend.text = TRUE
)
```

Interpretation:

The 100% stacked bar chart converts each region to the same total proportion of 100%. This makes it easier to compare the **relative product composition** of North, South, East and West.



5. Explain why one of these visualizations may be more appropriate than another for comparing regional proportions.

The **100% stacked bar chart** is more appropriate for comparing regional proportions because every region is represented as **100%**, allowing the relative contribution of each product to be compared directly.

The ordinary stacked bar chart is more useful for comparing **raw purchase totals**, while the pie chart is useful for showing the **overall product composition** but is less effective for comparing several regions simultaneously.

Conclusion:
The **100% stacked bar chart** provides the clearest comparison of product proportions across regions because it removes differences in total regional purchase counts.

Question 4 – Nested Proportions Using Treemap and Sunburst Charts

Dataset: Company_Sales.csv

Region	Category	Subcategory	Sales
North	Electronics	Laptop	52000
North	Electronics	Phone	38000
North	Furniture	Chair	22000
North	Furniture	Desk	18000
South	Electronics	Laptop	46000
South	Electronics	Phone	42000
South	Furniture	Chair	25000
South	Furniture	Desk	21000
East	Electronics	Laptop	58000
East	Electronics	Phone	35000

East	Furniture	Chair	19000
East	Furniture	Desk	23000
West	Electronics	Laptop	49000
West	Electronics	Phone	44000
West	Furniture	Chair	27000
West	Furniture	Desk	20000

1. Create a treemap showing Region → Category → Subcategory, with rectangle size representing Sales.

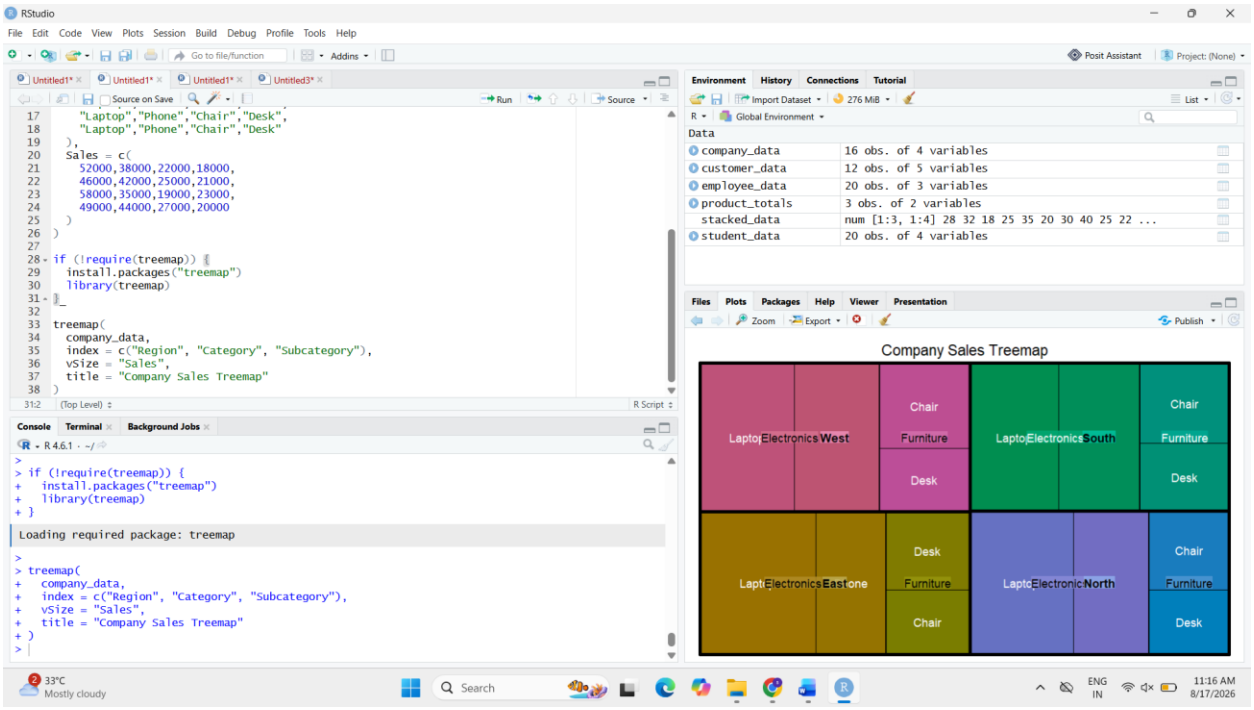
```
company_data <- data.frame(
  Region = c(
    "North","North","North","North",
    "South","South","South","South",
    "East","East","East","East",
    "West","West","West","West"
  ),
  Category = c(
    "Electronics","Electronics","Furniture","Furniture",
    "Electronics","Electronics","Furniture","Furniture",
    "Electronics","Electronics","Furniture","Furniture",
    "Electronics","Electronics","Furniture","Furniture"
  ),
  Subcategory = c(
    "Laptop","Phone","Chair","Desk",
    "Laptop","Phone","Chair","Desk",
```

```
    "Laptop","Phone","Chair","Desk",
    "Laptop","Phone","Chair","Desk"
  ),
  Sales = c(
    52000,38000,22000,18000,
    46000,42000,25000,21000,
    58000,35000,19000,23000,
    49000,44000,27000,20000
  )
)

if (!require(treemap)) {
  install.packages("treemap")
  library(treemap)
}

treemap(
  company_data,
  index = c("Region", "Category", "Subcategory"),
  vSize = "Sales",
  title = "Company Sales Treemap"
)
```

Interpretation:

The treemap represents the sales hierarchy using nested rectangles. The size of each

rectangle corresponds to the sales value, allowing the relative contribution of regions, categories and subcategories to be compared.



2. Create a sunburst chart representing the same hierarchy.

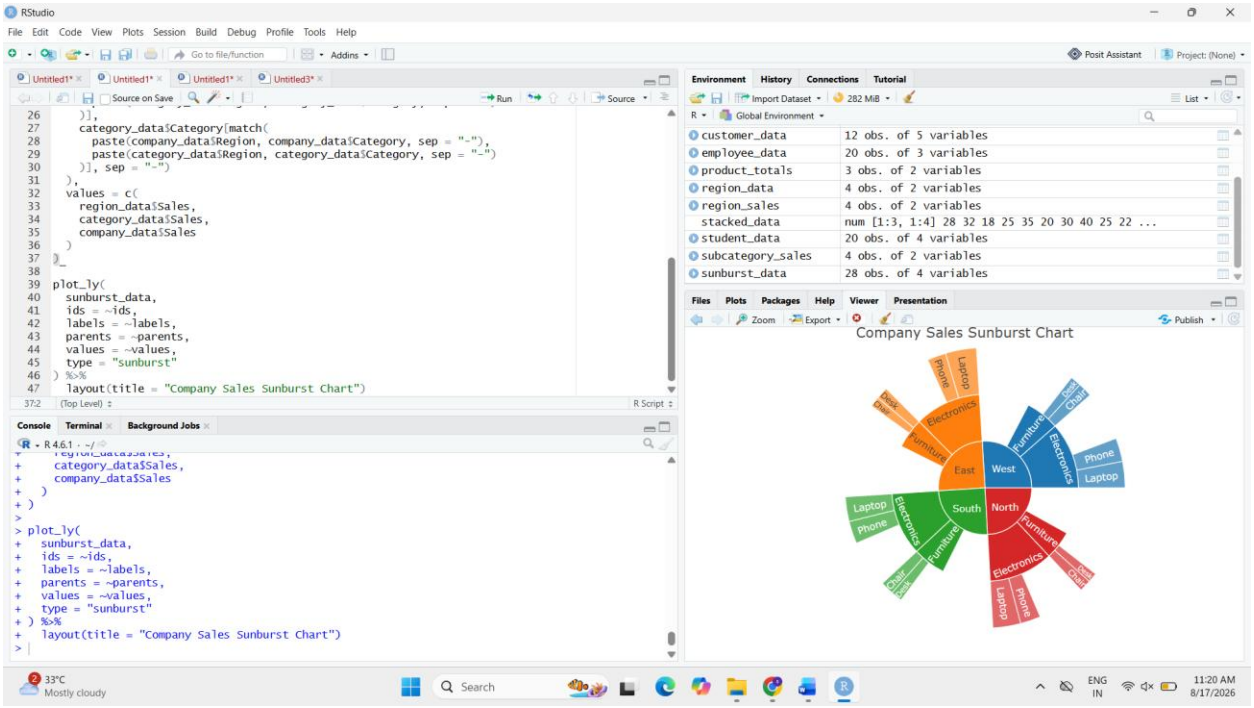
```
if (!require(plotly)) {
  install.packages("plotly")
  library(plotly)
}


plot_ly(
  company_data,
  ids = ~Subcategory,
  labels = ~Subcategory,
  parents = ~Category,
  values = ~Sales,
```

type = "sunburst"

)

Interpretation:

The sunburst chart represents the same hierarchical structure using concentric rings. The inner levels represent broader categories, while the outer levels represent subcategories.



3. Identify the largest region, category, and subcategory by sales.

region_sales <- aggregate(Sales ~ Region, company_data, sum)

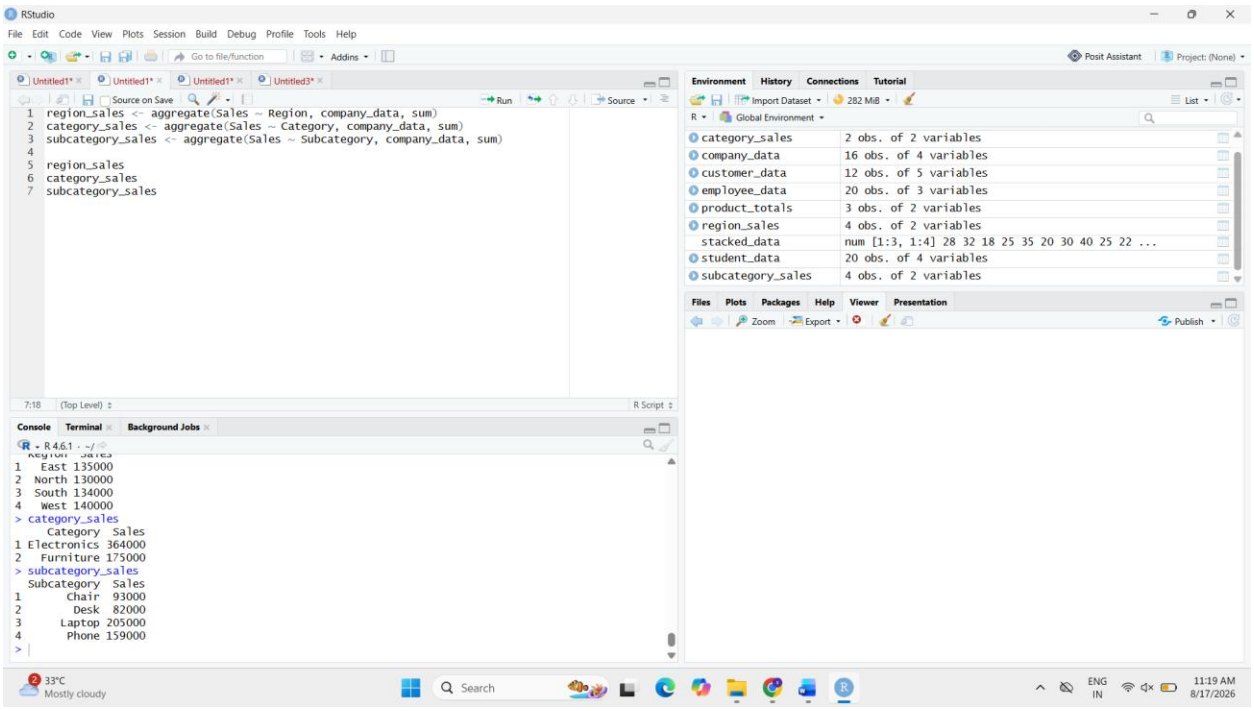
category_sales <- aggregate(Sales ~ Category, company_data, sum)

subcategory_sales <- aggregate(Sales ~ Subcategory, company_data, sum)

region_sales

category_sales

subcategory_sales



Result:

- **Largest Region:** East – **₹135,000**
- **Largest Category:** Electronics – **₹364,000**
- **Largest Subcategory:** Laptop – **₹205,000**

4. Compare the readability of the treemap and sunburst chart for identifying nested proportions.

The **treemap** is easier for comparing the relative sizes of sales because larger rectangles can be directly compared by area.

The **sunburst chart** is useful for understanding the hierarchical relationship between Region, Category and Subcategory because the levels are displayed as concentric rings.

Therefore, the **treemap is more readable for comparing sales proportions**, while the **sunburst is useful for understanding the hierarchy**.

5. State one potential limitation of each visualization.

- **Treemap:** Comparing similarly sized rectangles can be difficult because area is harder to judge precisely than length.

- **Sunburst:** Comparing segments across different rings can be difficult, especially when many categories or subcategories are present.

Conclusion:
The treemap provides an effective view of the relative sales proportions, while the sunburst gives a clearer representation of the hierarchical structure. The **East region, Electronics category, and Laptop subcategory** have the highest sales at their respective levels.

Question 5 – Flow-Based Proportions with Sankey Diagram and Parallel Sets

Dataset: Student_Career_Path.csv

Student_Group	Specialization	Internship	Placement_Sector	Students
G1	Data Analytics	Yes	IT	18
G2	Data Analytics	Yes	Finance	12
G3	Data Analytics	No	IT	8
G4	AI	Yes	IT	20
G5	AI	Yes	Healthcare	10
G6	AI	No	IT	6
G7	Cyber Security	Yes	IT	16
G8	Cyber Security	Yes	Finance	9
G9	Cyber Security	No	Government	7
G10	Cloud Computing	Yes	IT	15
G11	Cloud Computing	Yes	Finance	8

G12	Cloud Computing	No	Government	5

1. Create a Sankey diagram showing Specialization → Internship → Placement_Sector, with flow width based on Students.

```
if (!require(networkD3)) {
  install.packages("networkD3")
  library(networkD3)
}


career_data <- data.frame(
  Student_Group = c(
    "G1","G2","G3","G4","G5","G6",
    "G7","G8","G9","G10","G11","G12"
  ),
  Specialization = c(
    "Data Analytics","Data Analytics","Data Analytics",
    "AI","AI","AI",
    "Cyber Security","Cyber Security","Cyber Security",
    "Cloud Computing","Cloud Computing","Cloud Computing"
  ),
  Internship = c(
    "Yes","Yes","No",
    "Yes","Yes","No",
    "Yes","Yes","No",
    "Yes","Yes","No"
```

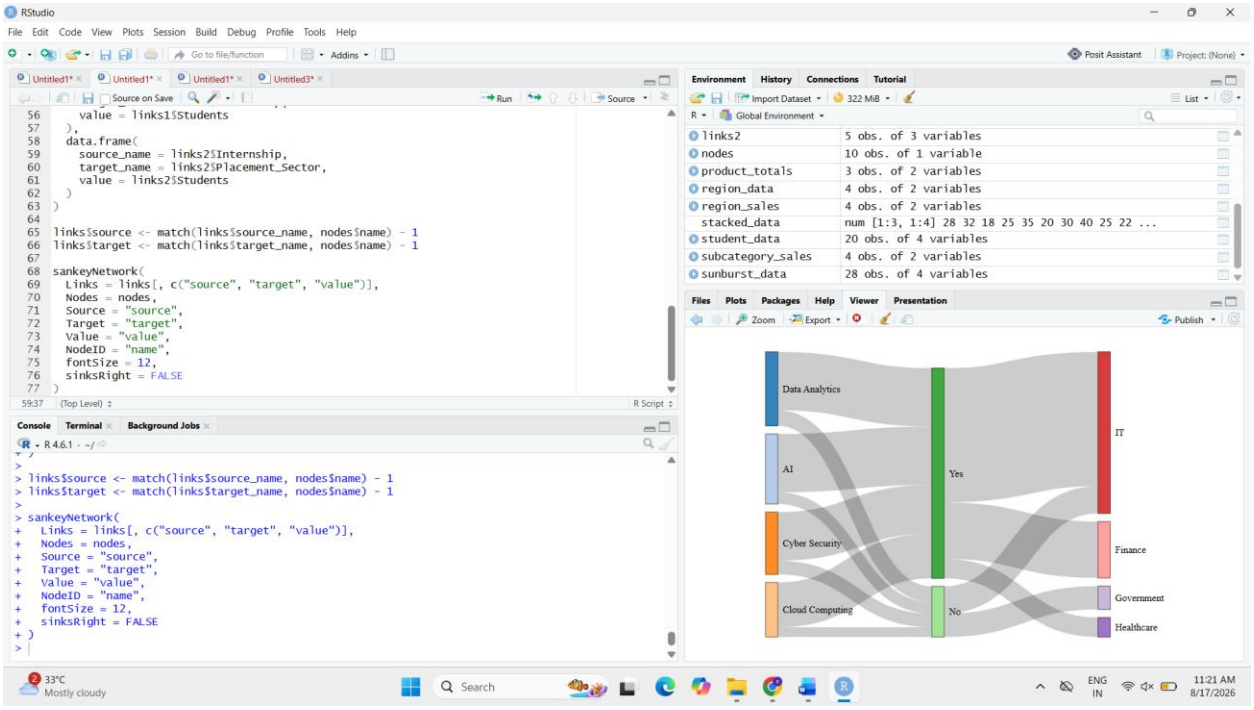
```
  ),
  Placement_Sector = c(
    "IT","Finance","IT",
    "IT","Healthcare","IT",
    "IT","Finance","Government",
    "IT","Finance","Government"
  ),
  Students = c(18,12,8,20,10,6,16,9,7,15,8,5)
)

nodes <- data.frame(
  name = unique(c(
    career_data$Specialization,
    career_data$Internship,
    career_data$Placement_Sector
  ))
)

links1 <- aggregate(
  Students ~ Specialization + Internship,
  data = career_data,
  sum
)
```

```
links2 <- aggregate(
  Students ~ Internship + Placement_Sector,
  data = career_data,
  sum
)

links <- rbind(
  data.frame(
    source_name = links1$Specialization,
    target_name = links1$Internship,
    value = links1$Students
  ),
  data.frame(
    source_name = links2$Internship,
    target_name = links2$Placement_Sector,
    value = links2$Students
  )
)

links$source <- match(links$source_name, nodes$name) - 1
links$target <- match(links$target_name, nodes$name) - 1

sankeyNetwork(
  Links = links[, c("source", "target", "value")],
```

```
  Nodes = nodes,
  Source = "source",
  Target = "target",
  Value = "value",
  NodeID = "name",
  fontSize = 12,
  sinksRight = FALSE
)
```

Interpretation:

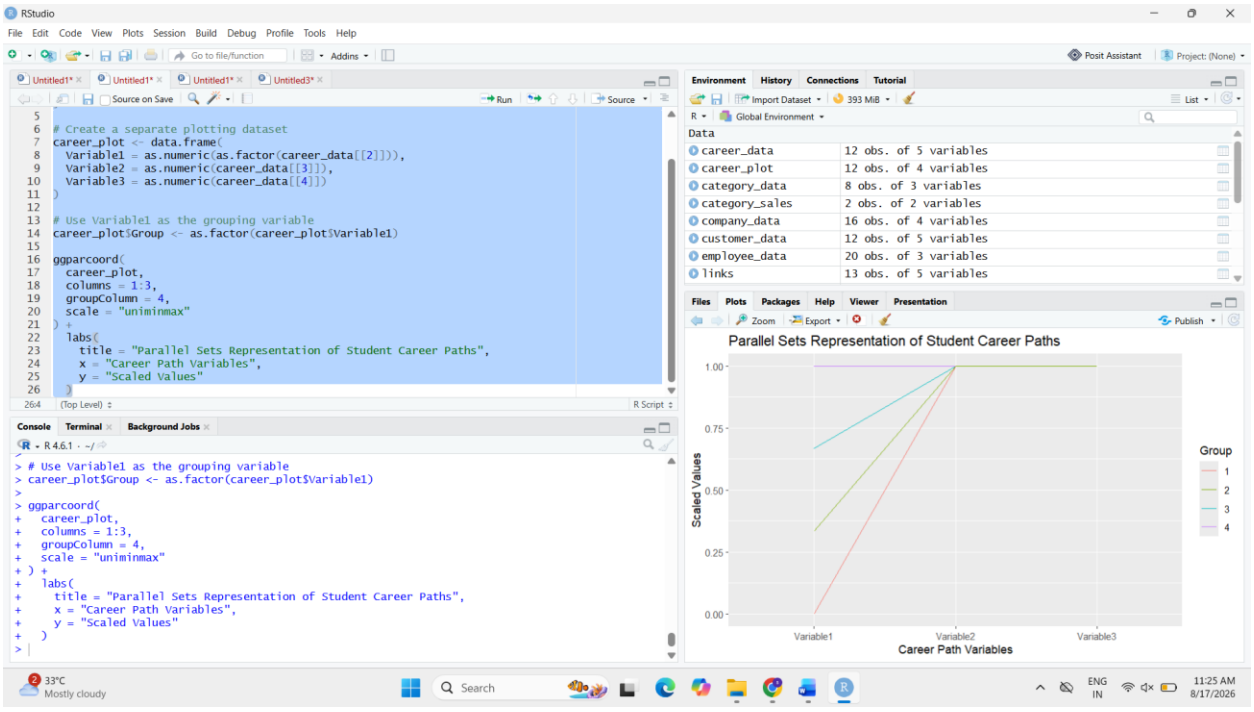
The Sankey diagram represents the movement of students from their specialization through internship status to placement sector. The width of each flow represents the number of students following that path.



2. Create a parallel sets visualization for the same categorical relationships.

```
if (!require(GGally)) {
  install.packages("GGally")
```

```
  library(GGally)
}

ggparcoord(
  career_data,
  columns = c(2, 3, 4),
  groupColumn = 2,
  scale = "uniminmax"
) +
  labs(
    title = "Parallel Sets Representation of Student Career Paths",
    x = "Career Path Variables",
    y = "Scaled Values"
  )
```



Interpretation:
The parallel visualization shows how students are distributed across specialization, internship status and placement sector. It allows relationships between the categorical variables to be examined simultaneously.

3. Identify the major flows and compare the placement patterns of students with and without internships.

Major flows:

- **AI → Yes → IT:** 20 students
- **Data Analytics → Yes → IT:** 18 students
- **Cyber Security → Yes → IT:** 16 students
- **Cloud Computing → Yes → IT:** 15 students
- **Data Analytics → Yes → Finance:** 12 students
- **AI → Yes → Healthcare:** 10 students

Students **with internships** have considerably larger flows into placement sectors, particularly **IT**. Students **without internships** have smaller flows: Data Analytics → IT has 8 students, AI → IT has 6 students, Cyber Security → Government has 7 students, and Cloud Computing → Government has 5 students.

Therefore, the dataset shows that students with internships have a **stronger representation in IT, Finance and Healthcare placements**, while students without internships have smaller placement flows, with some moving into Government.

4. Explain which visualization communicates the flow structure more clearly and why.

The **Sankey diagram** communicates the flow structure more clearly because the width of each connection directly represents the number of students. It clearly shows how students move from specialization to internship status and finally to placement sector.

The parallel sets visualization is useful for comparing multiple categorical relationships, but the individual flow quantities are less immediately clear.

Therefore, **Sankey is the more suitable visualization for communicating the flow structure**.

5. Discuss one way the chart could become misleading if proportions are represented using inappropriate scales, colors, or labels.

The chart could become misleading if **flow widths are not proportional to the actual number of students**. For example, giving a small flow the same visual width as a much larger flow could make their proportions appear equal.

Similarly, inappropriate colors or unclear labels could make different internship groups or placement sectors difficult to distinguish and lead to incorrect interpretation.

Conclusion:

The Sankey diagram clearly shows that the largest flows are associated with students who completed internships, particularly into the **IT sector**. Appropriate flow widths, colors and labels are essential to ensure that the visualization represents the actual student proportions accurately.